Universidad Distrital Francisco José de Caldas

School of Engineering

Systems Engineerings

Workshop 3.

Luis Sebastián Martínez Guerrero

Leidy Marcela Morales Segura

Supervisor: Carlos Andrés Sierra

Bogotá D.C.

July 5, 2025

Contents

Abstract

In the context of digital commerce, the ability to manage and analyze large volumes of information efficiently is fundamental for delivering responsive, scalable, and intelligent user experiences. This report presents the design of a database management system tailored to the operational and analytical needs of an e-commerce platform, in this case NexusBuy. The proposed architecture adopts a modular and distributed approach that separates transactional and analytical workloads, ensuring consistency, performance, and adaptability. The system integrates structured and unstructured data sources, supports real-time processing, and incorporates components for business intelligence and personalized user interactions. The design is driven by user-centric requirements and validated through modeled data flows, entity relationships, and query scenarios. Although conceived as a prototype for academic purposes, the solution reflects industry-aligned practices and demonstrates how database architecture can support the functional, non-functional, and strategic demands of modern e-commerce environments.

1 Introduction

In today's dynamic digital economy, the demand for robust, scalable, and intelligent e-commerce platforms is increasingly essential. E-commerce has evolved from simple online stores to complex ecosystems capable of handling millions of simultaneous transactions, personalized recommendations, and real-time business analytics. The foundation of these platforms lies in the critical need for a high-performance database management system capable of managing diverse types of data (structured, semi-structured, and unstructured) across multiple regions with minimal latency.

This project focuses on the design of a database management system for "NexusBuy," an e-commerce platform with a business identity under construction. While NexusBuy is inspired by the efficiency, scalability, and wide range of services offered by major industry leaders such as Mercado Libre (including user authentication, product listings, transaction processing, inventory updates, customer reviews, and promotions), our goal is to establish a database architecture that adapts to the needs and specifics of a new business.

Traditional monolithic database solutions are no longer adequate for addressing complex and highly demanding scenarios such as those posed by NexusBuy. Although its infrastructure is proprietary, it is essential that it be supported by data management systems capable of ensuring high availability, consistency, fault tolerance, and optimal response times.

The main objective of this project is to define a database architecture that not only supports NexusBuy's daily transactional operations but also enables advanced analytics and decision-making capabilities. This document presents a proposed database design and management approach that reflects the real needs of an e-commerce system of this magnitude. It includes a detailed set of functional and non-functional requirements derived from user stories and system expectations. The main challenges addressed include the need for high availability, rapid query responses, seamless data ingestion, real-time analytics, and secure data management. These aspects are critical to providing an optimal user experience and providing administrators, business analysts, and decision-makers with accurate and timely information.

To achieve these goals, the proposed system for NexusBuy will employ a database architecture that separates reads (queries) from writes (transactions), utilizing caching and asynchronous replication. Business intelligence will be integrated through data warehousing and visualization tools, enabling strategic insights from transactional and behavioral data. All of this will be justified by the anticipated data volumes and information flows between database components, defining a roadmap for a system designed not only to meet current business demands but also to adapt to NexusBuy's future growth and innovation.

See the workshop website for more instructions:
`https://github.com/SebasMa24/Course-DataBases2`

1.1 General Objective

To design and document a distributed database system for an e-commerce platform that ensures performance, security, and business insight.

1.2 Specific Objective

- Define functional and non-functional requirements.
- Establish a scalable and fault-tolerant architecture.
- Design an entity-relationship model and normalize the data.
- Implement support for role-based access and security.
- Enable integration with BI tools for strategic decision-making.

1.3 Background

The evolution of database technologies has reflected the exponential growth and increasing complexity of modern e-commerce platforms. Initially, most systems operated with centralized, monolithic databases and vertically scalable engines designed for smaller, isolated applications. These systems were sufficient during the early stages of digital commerce, offering simple and easy-to-control configurations. However, as user bases expanded and transaction volumes skyrocketed, the limitations of monolithic architectures became increasingly apparent. These include data processing bottlenecks, the inability to scale horizontally, and difficulties supporting multi-regional operations and real-time analytics.

In contrast, modern e-commerce ecosystems, like the aspiring NexusBuy, demand distributed, resilient, and intelligent database infrastructures. These systems must handle high-performance workloads, enable rapid query performance, and support concurrent access from millions of users worldwide. The shift from monolithic databases to distributed systems addresses these challenges by introducing capabilities such as horizontal scaling, data sharding, in-memory caching, and asynchronous replication. These issues, critical to scalability and availability, will be explored in detail in the Technical Decisions section of this project, where the choice of technologies will be justified later.

Mercado Libre, a leading e-commerce platform in Latin America, exemplifies this technological evolution. Initially built on traditional relational databases, the company has progressively adopted advanced data infrastructure strategies. Today, its architecture leverages a combination of microservices, distributed NoSQL and SQL databases, real-time data streaming (e.g., Apache Kafka), and cloud-native platforms to manage and scale its operations. These systems support vital platform features such as fraud detection, product recommendations, logistics optimization, and business intelligence.

A relevant comparison can be made between traditional and current database models in terms of scalability, fault tolerance, and analytical capabilities. While monolithic databases rely on vertical scaling and are prone to single points of failure, Mercado Libre's distributed system utilizes multi-region replication and elastic scalability, thereby improving availability and performance. This same philosophy will guide the design of NexusBuy, prioritizing the separation of reads and writes to optimize performance and resilience to failures.

Recent advances in unified architectures, such as the Lakehouse model, further support the integration of operational and analytical workloads. This model overcomes the limitations of isolated data systems by combining the reliability of data warehouses with the scalability of data lakes. Such solutions are increasingly being adopted in large-scale e-commerce environments to enable faster and smarter decision-making. The implementation of business intelligence at NexusBuy, using the collected and analyzed data, will be a project-wide issue, highlighting how database design enables these strategic capabilities.

In conclusion, the transition from monolithic databases to high-performance distributed platforms has become essential for scalability and competitiveness in e-commerce. The Mercado Libre case demonstrates how the adoption of modern data architectures supports innovation, improves user experience, and enables continued operational efficiency in a data-intensive environment. NexusBuy seeks to replicate and adapt these lessons learned to build a robust database infrastructure that will drive its growth and meet its ambitious business objectives.

1.4 Scope

The scope of this project encompasses the architectural design and documentation of a distributed database management system tailored for a large-scale e-commerce platform, taking Mercado Libre as a reference model. The system focuses on supporting core business functionalities such as user registration and secure authentication, product and inventory management, transaction processing, order tracking, and personalized product recommendations.

In addition, the system includes mechanisms for configuring promotional campaigns, implementing regional partitioning strategies, and enabling real-time data replication across multiple geographic regions (specifically Colombia, Mexico, and Argentina). These countries were selected for their strategic relevance in the Latin American e-commerce landscape, offering a diverse mix of user behavior, infrastructure maturity, and regulatory environments.

The design also incorporates caching techniques aimed at reducing latency, and business intelligence components that enable dynamic reporting, user behavior analysis, and decision-making support through data visualization tools.

Security is addressed through the use of role-based access control (RBAC), input validation, and encryption, ensuring data privacy and regulatory compliance.

This project is limited to the conceptual and logical design phases; it does not include integration with third-party APIs such as payment gateways or logistics providers. The outcome is a foundational blueprint for a scalable, fault-tolerant, and intelligent database system that meets the evolving needs of a modern e-Commerce platform, leveraging technologies such as SQL or NoSQL engines, cache systems, and other strategies that can help us.

1.5 Assumptions

During the research and design of the distributed database system for the e-Commerce platform, the following assumptions were made. The volume of data used in this phase was deliberately selected to reflect realistic usage conditions, based on a synthetic dataset representing 600,000 users, 48,000 sellers, 288,530 products, and 3 million orders. This dataset provides a statistically meaningful base to evaluate performance, scalability, and architectural decisions without overloading the system with unnecessary volume. The chosen size enables accurate estimation of growth rates, validation of partitioning strategies, and stress testing of transactional workloads, while remaining manageable for initial deployment and iterative development.

The geographic distribution of users across Colombia, Mexico, and Argentina was selected strategically due to their established digital commerce ecosystems, growing internet penetration, and cultural and economic diversity within Latin America. These countries provide a representative mix of regional behaviors, infrastructure maturity, and market size, making them ideal for testing distributed strategies such as regional partitioning, latency optimization, and localized data regulations.

- **Data Volume and Growth:** The platform is expected to handle an initial load of approximately 4,000 transactions and 320,000 user interactions per day. These estimates are derived from a predefined dataset simulating 600,000 registered users (distributed equally across Colombia, Mexico, and Argentina), 48,000 sellers, 288,530 products, and 3,000,000 historical orders, generating over 8.9 million order details. Based on this base load, a realistic annual growth rate of **25%** is projected for both transactions and interactions, in line with typical e-commerce scaling curves for emerging platforms.

- **Real-Time Requirements:** The system must support real-time data processing for features such as product recommendations, order tracking, and analytics dashboards to enhance user experience and enable dynamic decision-making.

- **User Behavior and Access Patterns:** It is assumed that users will exhibit typical e-commerce behavior, including activity spikes during promotions, flash sales, and holiday periods. Global concurrent access is also expected, especially in multilingual and multi-regional contexts.

- **Tools and Technologies:** The architecture assumes the use of scalable, open-source, and widely adopted technologies (e.g., SQL engines, NoSQL engines, container orchestration platforms) capable of supporting hybrid data models (relational and non-relational) as the system evolves.

- **Data Quality and Integrity:** Input data (products, reviews, orders, etc.) is assumed to be mostly valid. Automated validation mechanisms and constraints will manage exceptions and maintain referential integrity.
- **Business Scope and Constraints:** Although the NexusBuy business model is in progress, its database architecture references industry leaders like Mercado Libre for scalability. However, integration with external APIs (e.g., payment gateways, third-party logistics, fraud detection services) is excluded from this scope, which focuses solely on the internal logical and physical database design.
- **Partitioning and Regional Strategy:** The relational database is assumed to be partitioned by region (Colombia, Mexico, Argentina) to optimize query performance, ensure data locality, and enable horizontal scalability in a distributed environment.
- **NoSQL Components and Hybrid Model:** Non-relational components such as product images, product discounts, Q&A, chat, recommendations, and publicity are assumed to be stored in NoSQL databases. The hybrid model supports flexibility for unstructured or semi-structured data.
- **Synthetic Dataset Validity:** All estimations and evaluations are based on a predisposed synthetic dataset constructed to mirror realistic usage patterns. While not derived from production, it serves as a reliable baseline for performance benchmarking and growth projection.
- **Monitoring and Observability:** It is assumed that the system will incorporate logging, metrics collection, and performance monitoring to evaluate health and usage over time, which is critical for scaling and troubleshooting.

2 Description of the NexusBuy Database Management System

NexusBuy's database management system is conceived as an interconnected set of elements, designed to ensure an efficient, reliable, and timely flow of information. The key components, their interrelationships, and the critical points that must be managed to ensure the platform's performance and reliability are detailed below.

2.1 System elements

The system elements are grouped into four key dimensions: Users, Data, Processes, and Technology Resources (backend modules, databases, API Gateway, and analytics tools), each fulfilling a specific function within the system and essential for its proper functioning in the context of NexusBuy.

1. Users
   Actors who interact directly or indirectly with the system, whose needs drive the database design:
   - **Buyers:** End users who search, browse, and purchase products through the platform. Their interaction generates a high volume of browsing, search, and transaction data.
   - **Sellers:** Users who list, manage, and sell products. Their actions involve inventory updates, order management, and communication with buyers, generating transactional and product management data.
   - **Technical Users:** Database administrators, developers. Their roles are not directly reflected in customer-facing user stories, but they are critical to the maintenance and evolution of the system.
   - **Functional Users:** Analysts, sales managers, product supervisors, marketing analysts, operations managers. Their needs are translated into the functional and non-functional requirements of the system, driving the database's query and analysis capabilities.
2. Data:
   Data is the primary input and the backbone of all interrelationships. For NexusBuy, it includes:
   - **Structured data**: Information organized in a relational schema, such as users, products, orders, payments, inventory. These are essential for transactional operations.

- **Unstructured/semi-structured data**: More flexible and diverse information, such as reviews, user searches, platform interactions (clicks, views), and questions and answers. These are crucial for personalization and behavioral analysis.
- **Simulated data**: CSV and JSON files generated to represent the behavior of a real-world environment. This data will be the basis for testing and validating the database design in the initial phases of the project.

3. Processes:
   A set of automated activities that transform data into useful information for NexusBuy:

   - **Data ingestion**: Periodic reading of synthetic files and capture of real-time event streams.
   - **Transformation**: Data cleansing, normalization, and enrichment to ensure quality and consistency before storage or analysis.
   - **Loading**: Insertion of transformed data into operational and analytical databases.
   - **Queries**:Transactional operations (e.g., stock verification, order entry) and analytical operations (e.g., sales reports).
   - **Aggregation and recommendation generation**:Application of business logic to extract value from the data, feed BI dashboards, and custom recommendation engines.

4. Backend Functional Modules:
   System components that organize business logic and interconnect with the database.

   - **User management**
   - **Product management**
   - **Order and payment management**
   - **Review and generated content management**
   - **Personalized recommendations module**

5. Databases
   The logical infrastructure where information is stored according to its nature and purpose in NexusBuy.

   - **Relational database (transactional SQL)**: Stores structured operational data with a focus on integrity and consistency.
   - **Document database (analytical NoSQL)**: Contains more flexible, unstructured or semi-structured data used in analysis and recommendations (e.g., user interaction data, reviews).
   - **Caching system**:Accelerates access to high-demand data, reducing the load on primary databases and improving response latency for users.

6. API Gateway
   Entry point for simulated external requests, connecting the system to its environment.

   - RESTful services for interaction with simulated external applications.
   - Authentication and validation mechanisms to ensure secure interactions.
   - Efficient routing to the corresponding backend modules.

7. Analytics Tools (BI)
   Query and display module for NexusBuy functional users.

   - Dashboards displaying key data.
   - Key performance indicators to monitor business performance.
   - Automated or on-demand reports for in-depth analysis.

2.2 Interrelationship between components

In a complex system like the one proposed for an e-commerce platform, the elements do not operate in isolation, but are strongly interconnected. This functional interdependence ensures that data flows coherently and efficiently from its point of origin to its final use, whether operational or analytical. Each component fulfills a specific role, but its effectiveness depends on proper functioning and synchronization with the other elements.

- Coordination between processes and data
  One of the fundamental links in the system is the relationship between processes and data. Synthetic data, which simulates user, product, and transaction information, enters the system as flat files or simulated information streams. This data, on its own, is worthless until it is transformed through processes defined in the ETL (Extract, Transform, Load) pipeline. The interaction between these processes and data sources is critical: an error in the transformation process can alter the expected structure, compromising the integrity of the relational model or generating inconsistencies in the analytical repositories. Thus, the NexusBuy system depends on a precise and controlled processing chain, where each stage feeds into the previous one and prepares the next.

- Communication between backend modules
  The modules that comprise NexusBuy's backend (users, products, orders, payments, reviews) share common entities and constantly communicate with each other. For example, the orders module relies on the products module to verify availability, and on the users module to associate the purchase with a customer. This cross-relationship implies a need for referential integrity and real-time data consistency. Therefore, the transactional database (SQL) acts as a hub that connects all these modules through well-defined relationships. A change in one module can have an immediate effect on the others, requiring robust design of validations and business rules.

- Synchronization between transactional and analytical storage

  Another key interrelationship exists between operational and analytical storage systems. While the relational database stores the structured information needed for real-time operations, the document repository (NoSQL) enables the exploration of behavioral data and informal interactions. The ETL system acts as a bridge between the two worlds, ensuring that operational data is consistently extracted, transformed, and loaded into the analytical system. This connection is vital to avoid inconsistencies in BI dashboards or misalignment between what happens in the operation and what is analyzed in NexusBuy metrics.

- External Environment and Interoperability
  Finally, the NexusBuy system is not isolated: it interacts with a simulated external environment, composed of REST clients, data sources, and test applications. These external inputs interact with the system through the API Gateway, whose function is to mediate between external requests (simulated, which would emulate the interaction of a frontend or other systems) and the internal logic. Another essential interrelationship arises here: the API not only exposes services but also depends on the backend's response and the status of the databases to deliver accurate and timely information. If any of the subsystems fails, the external interface will reflect that problem, demonstrating the systemic and connected nature of the design.

2.3 Inputs, processes and outputs

NexusBuy's database management system can be viewed as a system with well-defined inputs, processes, and outputs, ensuring the transformation of raw data into valuable information.

- Inputs: The system ingests files in structured and semi-structured formats (CSV and JSON), which contain synthetic data that simulates user behavior, products, orders, and advertising campaigns. Additionally, the system receives external requests through programming interfaces (REST APIs), which represent customer interactions or automated applications. Finally, internally generated data is included, such as browsing logs, user searches, comments, and ratings.

- Processing: Data processing is performed through an automated workflow that includes validation, transformation, and loading (ETL). This process organizes the information so that it can be distributed across two types of storage: one oriented toward transactional operations and the other toward behavioral analysis. During this stage, indexing, partitioning, and caching strategies

are implemented to optimize query performance. Derived information such as key performance indicators, search-to-purchase conversions, and review rankings are also calculated. Part of the processing is used to feed recommendation modules that contribute to the personalization of the user experience.

- Outputs: The system's outputs consist of pre-structured data, ready to be used by different NexusBuy analytics areas. These include sales reports, consumer behavior, product trends, and campaign results. This information is presented through dashboards that allow strategic stakeholders to make evidence-based decisions. Additionally, the system provides near-real-time responses to API requests, including queries about product availability, personalized recommendations, and key metrics.

2.4 Critical System Points

There are critical points that can compromise the performance and reliability of NexusBuy's database system, including:

- High Concurrency in Read Queries: Especially for popular data such as featured products or updated stock. Without adequate caching mechanisms, these queries could overload the transactional engine, generating unacceptable latencies.
- Constant Data Ingestion: The system must process multiple synthetic files and event streams throughout the day. Poor management of ETL processes can lead to inconsistencies, redundancies, or data loss.
- Handling Complex Analytical Queries: Such as those related to performance by region or campaign analysis. These queries, which involve large volumes of historical data and aggregated calculations, can become slow if optimized structures for analysis are not designed.
- Integrity between Operational and Analytical Data: The transformation and loading process must maintain consistency between the transactional records used in operations and those used in the analysis. Otherwise, discrepancies could arise that affect the indicators displayed or lead to errors in decision-making.

3 Model canvas

The Canvas Model is a fundamental strategic tool that allows us to comprehensively and concisely visualize the nine key building blocks of a business. In the context of this e-commerce platform, the Canvas helps us clearly define the value proposition offered to users, identify the target customer segments, establish key relationships with these customers, and determine the channels through which we interact. It also details the essential activities and resources for operations, the strategic partners that contribute to the ecosystem, and, crucially, the cost structure and revenue streams that sustain the project's economic viability. This holistic view ensures that the technological architecture is directly aligned with the business objectives and strategy.

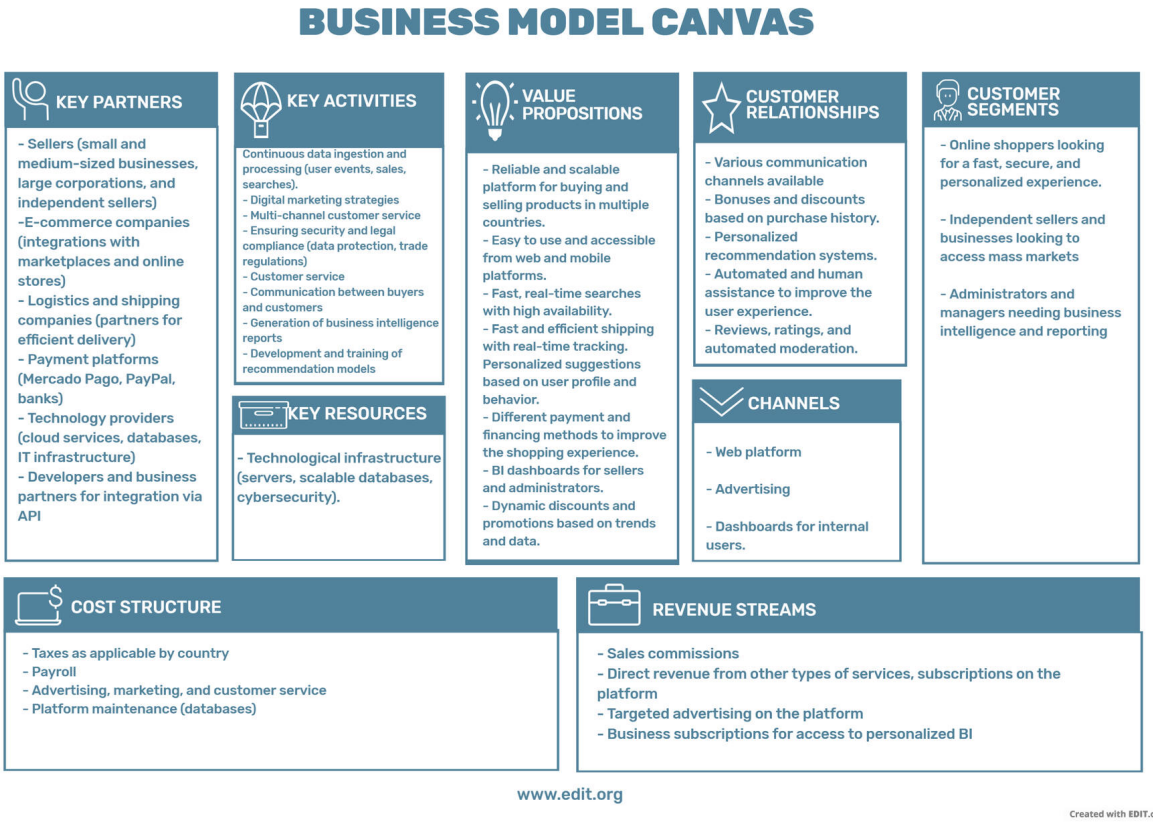

Figure 1: Business model canvas

4 User stories

User Stories are an agile tool that allows us to understand and document system functionalities from the perspective of the different actors who will interact with it. Each story is formulated in a simple and concise manner, following the format "As [role], I want [action], so that [benefit]." This allows us to capture the specific needs of buyers, sellers, administrators, and other roles, ensuring that development focuses on delivering real value to each type of user. Additionally, each story includes detailed acceptance criteria that define when a feature is considered complete and correct, facilitating its validation. This user-centered approach is vital to building an intuitive platform that truly solves its users' problems and desires.

4.1 Role: User

Title: Buyer Email Registration Priority: High
User story: As a Buyer I want register with my email so what i can access the platform easily
Acceptance Criteria: Given: A user wants to register on the platform. When: The user enters their email address and other required registration data, and the system attempts to validate the email. Then: The system allows registration using the email address. Email format validation is required. An account confirmation email is sent for account activation.

Title: Product Search with Filters Priority: High
User story: As a Buyer I want search for products with filters so what I can find what I need quickly
Acceptance Criteria: Given: I am on the product search page. When: I apply filters by category, price range, or rating. Then: Search results update to show only products that meet the selected filters. Search response time is less than 1 second. The search event is logged in the system.

Title: Personalized Product Recommendations Priority: Medium
User story: As a Buyer I want to see personalized recommendations So that I discover relevant products
Acceptance Criteria: Given: I am Browse the platform or logged into my account. When: The system shows me product recommendations. Then:I see at least 5 suggested products that are relevant to me. Recommendations update dynamically based on my browse and purchase history.

Title: Pay with Different Methods Priority: High
User story: As a Buyer I want to pay with different methods So that the purchase is more convenient
Acceptance Criteria: Given: I have reached the payment process for my purchase. When: I select a payment method. Then: I can choose to pay by card, bank transfer, or cash on delivery. Payment confirmation is automatic once the transaction is complete. Real-time error handling is provided, and I am notified of any payment issues.

Title: Leave Reviews for Purchased Products Priority: Medium
User story: As a Buyer I want to leave reviews for purchased products So that I can help other buyers
Acceptance Criteria: Given:I have purchased a product and it has been delivered. When: I attempt to leave a review for that product. Then: I can only leave reviews for products I have previously purchased. I can rate the product with a minimum of 1 star and a maximum of 5 stars. The review is automatically moderated before being published.

4.2 Role: Seller

Title: Create Detailed Product Listings Priority: High
User story: As a Seller I want to create detailed product listings So that I attract more buyers
Acceptance Criteria: Given: I am creating a new product listing. When: I enter the product information. Then: I can complete minimum fields like title, description, price, and stock. I can upload up to 5 images per product.

Title: Monitor Product Statistics Priority: Medium
User story: As a Seller I want to monitor my product statistics So that I can optimize their performance
Acceptance Criteria: Given: I access my seller dashboard. When: I review my product statistics. Then:I see metrics like views, sales, reviews, and stock. I can filter statistics by date and category. Statistics are visualized on a dashboard.

Title: Receive Low-Stock Notifications Priority: Medium
User story: As a Seller I want to receive notifications about low-stock products So that I can restock them
Acceptance Criteria: Given: I have products in my inventory. When: A product's stock falls below a configured threshold. Then: I receive a notification via email and on the platform. The low-stock threshold is configurable per product.

Title: Apply Temporary Promotions Priority: Medium
User story: As a Seller I want to apply temporary promotions So that I can boost sales
Acceptance Criteria: Given: I have active products in my store. When: I configure a promotion. Then: I can select products in batch for the promotion. I can define a start and end date for the promotion. I can visualize the sales impact of the promotion.

Title: Manage User Questions Priority: High
User story: As a Seller I want to manage user questions So that I build trust with buyers
Acceptance Criteria: Given: A buyer asks a question about one of my products. When: I receive a new question. Then: I am notified immediately of the new question. A response time of less than 24 hours is recommended.

4.3 Role: Administrator

Title: Manage Product Categories Priority: High
User story: As a Administrator I want to manage product categories So that the platform stays organized
Acceptance Criteria: Given: I need to organize products on the platform. When: I manage categories. Then: I can create, edit, and delete categories. Changes to categories have an immediate impact on search filters.

Title: Access Real-Time Global Platform Metrics Priority: High
User story: As a Administrator I want to access real-time global platform metrics So that I can proactively identify and address performance issues
Acceptance Criteria: Given: I need to understand the overall platform performance. When: I review global metrics. Then: I can see reports by region, day, and category. Growth charts and alerts are displayed.

Title: Review and Moderate User-Generated Content Priority: High
User story: As a Administrator I want to review and moderate user-generated content So that the platform maintains high-quality listings and trustworthy reviews
Acceptance Criteria: Given: There is user-generated content (e.g., reviews, descriptions). When: I need to moderate this content. Then: I have a list of reported posts. I can perform actions such as approving, hiding, or deleting content. The responsible moderator for each action is logged.

5 Requirements Documentation

The precise definition of functional and non-functional requirements is the foundation for developing any robust and efficient system. Functional requirements specify the capabilities and behaviors the system must exhibit to meet user and business needs, such as user management, filtered product searches, review publishing, or inventory management. Non-functional requirements, on the other hand, describe the system's qualities, such as its performance (response times), scalability (ability to handle large

volumes of data and users), security (protection of sensitive data), availability (24/7 operation), and maintainability. Correctly identifying and prioritizing both types of requirements is crucial to ensuring that the platform not only meets its operational objectives but is also sustainable and adaptable over the long term.

5.1 Functional Requirements

Table 1: Functional Requirements of the E-commerce Platform

ID	**Requirement Name**	**Description**	**Required Inputs (Fields)**
RF01	User Registration	The system must allow user registration with secure authentication.	Full Name, Email, ID Number, Address, Phone Number, Password
RF02	Product Search	The system must allow product searches using quick filters (category, price, rating).	Keyword, Category, Minimum Price, Maximum Price, Minimum Rating
RF03	Personalized Recommendations	The system must display recommendations based on user behavior.	User ID, Browsing History, Purchase History
RF04	Multiple Payment Methods	The system must allow payments via card, transfer, or cash on delivery.	Order ID, Payment Method, Card/Bank Info, Shipping Address
RF05	Publish Reviews	The system must allow reviews to be published only for purchased products.	User ID, Product ID, Rating, Comment
RF06	Create Listings	The system must allow products to be published with details and images.	Product Name, Description, Price, Stock, Images, Category
RF07	Product Statistics	The system must display statistics on views, sales, reviews, and stock.	Seller ID, Product ID, Date Range
RF08	Low Stock Alerts	The system must send notifications when stock is low.	Product ID, Threshold, Current Stock
RF09	Promotion Management	The system must allow creation of promotions with rules and duration.	Products, Discount, Start Date, End Date, Conditions
RF10	QA Management	The system must allow management of questions and answers in listings.	Product ID, Question ID, Response Text
RF11	Multi-Region Replication	The system must allow data replication across regions with low latency.	Regions, Cluster Configuration
RF12	Category Management	The system must allow creation/modification of categories visible in filters.	Name, Parent Category, Description
RF13	Metrics by Region and Category	The system must display segmented key performance indicators.	Date, Region, Category, Metric Type
RF14	Content Moderation	The system must allow moderation of listings and reviews with traceability.	Content ID, Action, Justification, Moderator ID
RF15	Sales KPIs Dashboard	The system must display key sales indicators in real time.	Date, Region, Category, KPI

End of table

5.2 Non- functional Requirements

Table 2: Non-Functional Requirements of the E-commerce Platform

ID	Requirement Name	Description
NFR01	Big Data	The system must store and process large volumes of data: 600,000 users, 48,000 sellers, 288,530 products, 3 million orders and 8.9 million order details. It must support both structured (SQL) and unstructured (images, Q&A, reviews) data.
NFR02	Fast Query Response	The system must ensure that product searches and order queries return results in less than 1 second, despite the volume of 320,000 daily user interactions. Indexing and caching strategies must be implemented for optimal query performance.
NFR03	Data Ingestion	The system must support ingestion of at least 4,000 transactions per day and 320,000 user interactions, allowing for batch uploads (e.g., product imports) and real-time streaming (e.g., user events), with reprocessing support in case of failure.
NFR04	Business Intelligence	The system must provide real-time analytics dashboards for sellers and administrators, derived from the 3 million historical orders and interaction logs, enabling insight into product performance, sales, inventory, and user activity across the 3 countries.
NFR05	Multi-Location Access	With users distributed in Colombia, Mexico, and Argentina, the system must replicate and route data regionally to reduce latency, using proximity-based load balancing to ensure efficient access regardless of user location.
NFR06	Recommendation System	The system must generate personalized recommendations for buyers using their interaction history (from 320,000 daily actions) and purchase records (3 million orders), updating suggestions dynamically during each session.
NFR07	High Availability	The system must provide 99.9% uptime (44 minutes downtime/month) using automatic replication and failover between regional instances. This ensures uninterrupted access during cross-country usage and time zone differences.
NFR08	Scalability	The system must handle the current simulated load efficiently, including 10,000 concurrent users, without needing major architectural changes. Horizontal scalability should be supported in case load balancing across regions is required.
NFR09	Security	The system must provide data encryption at rest and in transit, implement role-based access control (RBAC), and audit critical operations to prevent vulnerabilities.
NFR10	Maintainability	The system must be modular to allow updates without interruption, include detailed technical documentation, and support automated deployment and monitoring.
NFR11	Cost Optimization	The system must optimize infrastructure usage through auto-scaling, cold storage for historical data, and strategies to avoid overprovisioning.

End of table

6 ER Model

The Entity-Relationship (ER) Model is a conceptual representation of the database structure, fundamental to the design and implementation of an information system. This model identifies key business entities (such as Users, Products, Orders, Reviews, etc.), their attributes (the properties that describe each entity), and the relationships between them. The steps used to define the model are presented below.

6.1 Components

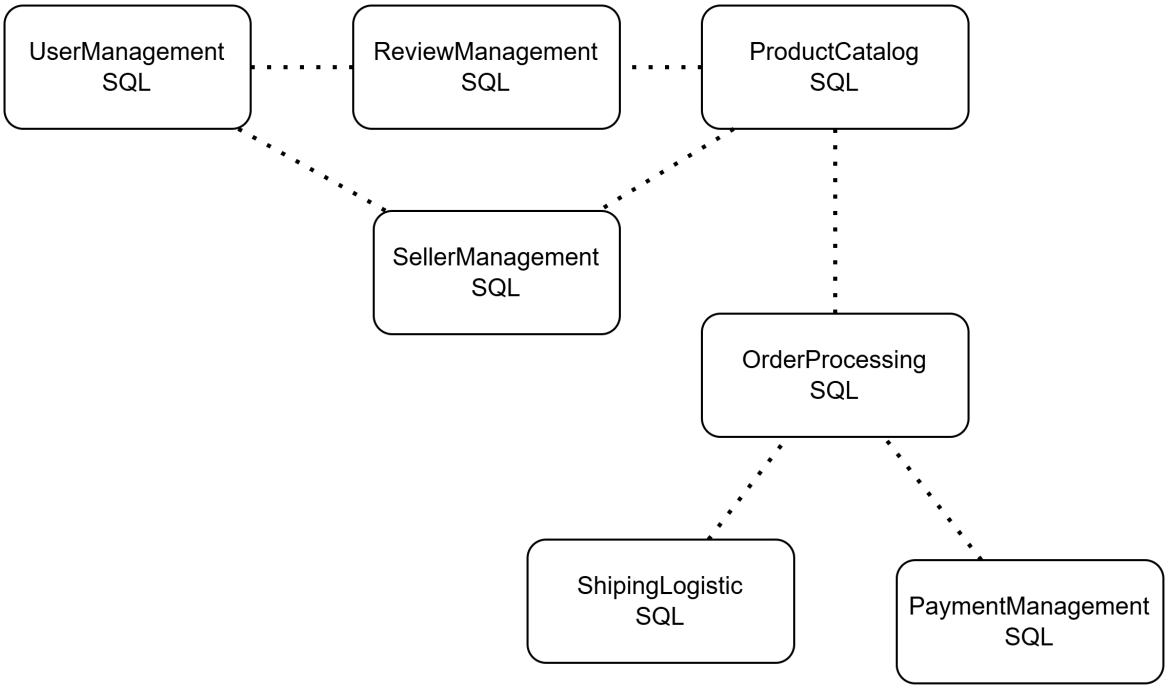

Figure 2: Components

6.2 Relationships

1. **UserManagement** has a relation with:
    a. SellerManagement, if the user is a store owner.
    b. OrderProcessing, since users place orders.
    c. ReviewManagement, since users write reviews.
2. **ProductCatalog** has a relation with:
    a. SellerManagement, since products belong to stores.
    b. OrderProcessing, since orders contain products.
    c. ReviewManagement, since products receive reviews.
3. **OrderProcessing** has a relation with:
    a. PaymentManagement, to register the transaction of the order.
    b. ShippingLogistic, to handle the delivery of the order.

6.3 Entities

- **UserManagement**
  - E1. Users
  - E2. Roles
  - E3. Addresses
- **SellerManagement**
  - E4. Stores
- **ReviewManagement**
  - E5. Reviews
- **ProductCatalog**
  - E6. Products
  - E7. Categories
- **OrderProcessing**
  - E8. Orders
  - E9. OrderDetails
  - E10. Coupons
- **ShippingLogistic**
  - E11. Shipments
  - E12. ShipmentStatuses
- **PaymentManagement**
  - E13. Payments
  - E14. PaymentMethod
  - E15. PaymentStatuses

6.4 Attributes per Entity

Table 3: Entities of the E-commerce Platform (SQL only)

ID	Entity Name	Attributes
E1	Users	id, name, email, password, phone, birthday, region, createdAt
E2	Roles	id, nameRole
E3	Addresses	id, userId, address, city, zipCode, country, region
E4	Stores	id, userId, name, isOfficial, region, createdAt
E5	Reviews	id, userId, productId, rating, comment, region, createdAt
E6	Products	id, storeId, categoryId, title, description, price, stock, region, createdAt
E7	Categories	id, name, description
E8	Orders	id, userId, couponId, totalPrice, region, createdAt
E9	OrderDetails	id, orderId, productId, quantity, unitPrice, region
E10	Coupons	id, code, valuePercentage, valueFixed, maxUsage, usageCount, expirationDate, isActive

(Continued on next page)

ID	Entity Name	Attributes
E11	Shipments	id, orderId, trackingNumber, carrier, shipmentStatusId, shippedAt, deliveredAt, region
E12	ShipmentStatuses	id, statusName, description
E13	Payments	id, orderId, amount, paymentMethodId, paymentStatusId, region, createdAt
E14	PaymentMethod	id, methodName
E15	PaymentStatuses	id, statusName, description

End of table

6.5 Entity Relationship Matrix (SQL Entities Only)

E\E	E1	E2	E3	E4	E5	E6	E7	E8	E9	E10	E11	E12	E13	E14	E15
E1	–	x	x	x	x			x					x		
E2	x	–													
E3	x		–												
E4	x			–		x									
E5	x				–	x									
E6				x	x	–	x	x	x						
E7						x	–								
E8	x					x		–	x	x	x		x		
E9						x		x	–						
E10								x		–					
E11								x			–	x			
E12											x	–			
E13	x							x					–	x	x
E14													x	–	
E15													x		–

Table 4: Complete Entity Relationship Matrix for SQL Components (E1–E15)

6.6 Relation Types

Table 5: Relation Types in the SQL-based E-commerce Model

From Entity	To Entity	Cardinality	Description
E1. Users	E2. Roles	1 : N	Each user is assigned one role. A role may be shared by many users.
E1. Users	E3. Addresses	1 : 0..N	A user may have multiple addresses. Each address belongs to one user.
E1. Users	E4. Stores	0..1 : 1	A user may own one store. Each store belongs to one user.
E1. Users	E5. Reviews	1 : 0..N	A user can post multiple reviews. Each review is written by one user.
E1. Users	E8. Orders	1 : 0..N	A user can place multiple orders. Each order belongs to one user.
E4. Stores	E6. Products	1 : 0..N	A store can have many products. Each product belongs to one store.
E6. Products	E7. Categories	N : 1	Each product belongs to one category. A category can contain many products.
E6. Products	E5. Reviews	1 : 0..N	A product can have multiple reviews. Each review is about one product.
E6. Products	E9. OrderDetails	1 : 0..N	A product can appear in multiple order details. Each detail references one product.
E8. Orders	E9. OrderDetails	1 : 1..N	Each order must have at least one product. Each detail is part of one order.

(Continued on next page)

From Entity	To Entity	Cardinality	Description
E8. Orders	E10. Coupons	0..1 : N	An order may use one coupon. A coupon can apply to many orders.
E8. Orders	E11. Shipments	1 : 1	Each order has one shipment. Each shipment belongs to one order.
E8. Orders	E13. Payments	1 : 1	Each order has one payment. Each payment is linked to one order.
E11. Shipments	E12. ShipmentStatuses	1 : N	Each shipment has one status. A status can apply to many shipments.
E13. Payments	E14. PaymentMethod	1 : N	Each payment uses one method. A method may be used in many payments.
E13. Payments	E15. PaymentStatuses	1 : N	Each payment has one status. A status can apply to many payments.
			End of table

6.7 First Model

First Model only with entities and relationships

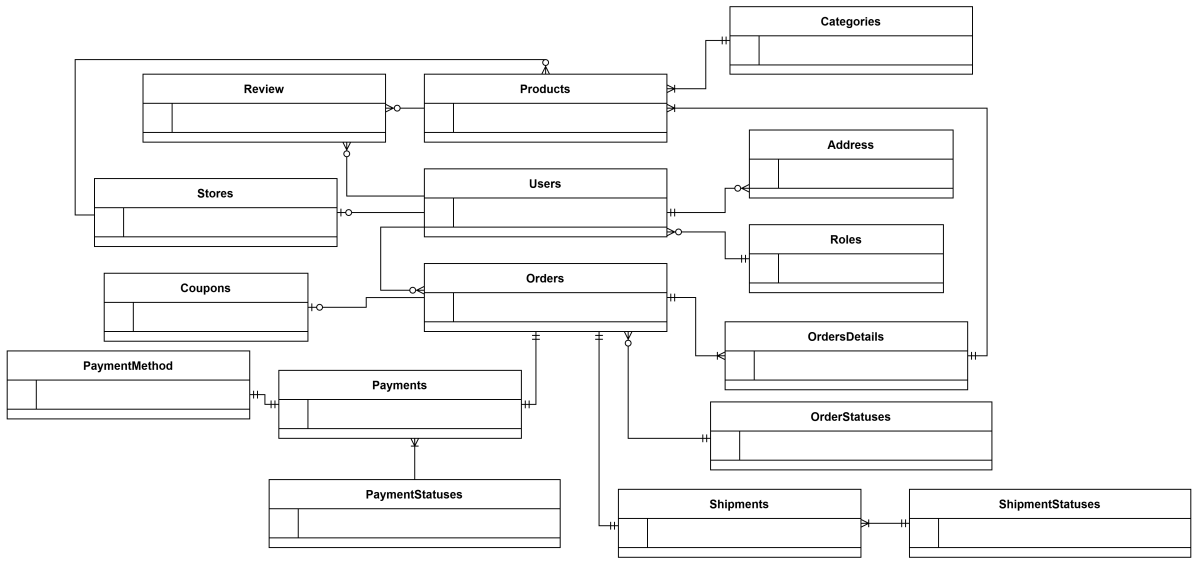

Figure 3: First model ER

6.8 Second Model

Second Model with attributes and data types

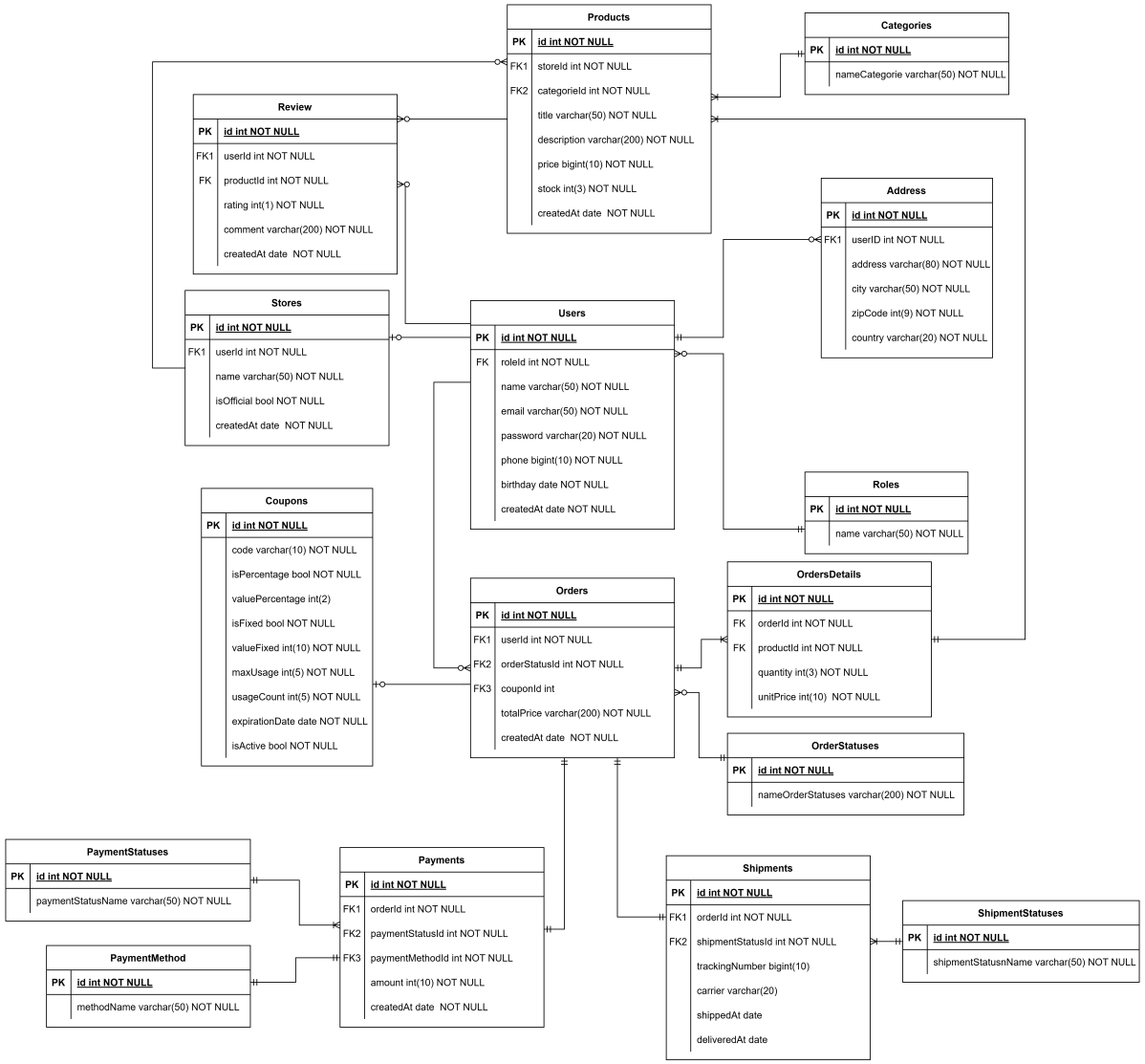

Figure 4: Second model ER

7 Database Architecture

NexusBuy's database architecture is designed to handle an initial volume of approximately 4,000 transactions and 320,000 user interactions per day, with a projected annual growth of 25%. This workload drives the adoption of a distributed and modular design, enabling horizontal scalability to accommodate increased operations. Targeted Read-Write Separation is critical; this strategy is crucial given that read operations, such as product searches or review viewing, are significantly more frequent than write operations (creating an order or updating stock), and require extremely low latency, with a response time of ¡1 second. Directing critical transactions, such as orders and payments, to integrity-optimized relational databases, while handling high-volume queries, which experience significant spikes in activity during promotions and sales events, with caching systems and NoSQL databases, maximizes performance and concurrency. This distinction optimizes speed, as reads don't compete for resources with intensive writes, avoiding bottlenecks and ensuring the database maintains high performance even under extreme loads. Additionally, regional data partitioning and replication improve latency and availability for globally distributed users, ensuring efficient and consistent access across millions of users and transactions.

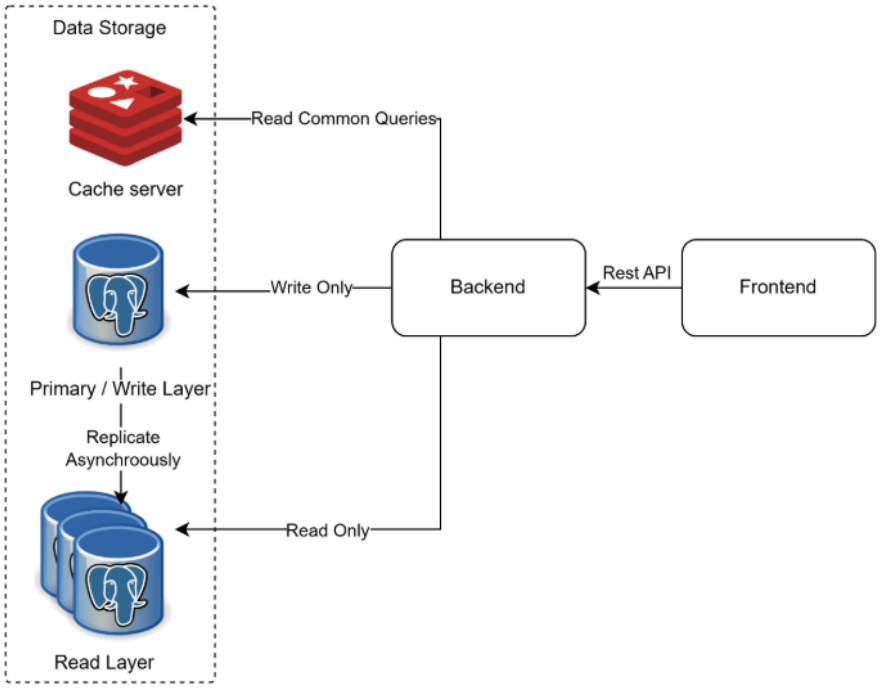

Figure 5: Database architecture

Main Components

1. **Write Layer**

   Technology: PostgreSQL.

   Responsibility: Write operations like `INSERT`, `UPDATE`, `DELETE`.

2. **Read Layer**

   Technology: PostgreSQL.

   Responsibility: Read operations like `SELECT`, and horizontally scalable replicas under load.

3. **Cache Layer**

   Technology: Redis.

   Use pattern: The application searches first in Redis. If the data doesn't exist, it queries the database. All writes to the primary database are also written to Redis.

   Data in cache:
   - Popular products: saving titles, prices, and discounts.
   - Daily offers: saving titles, prices, and discounts.
   - User data (tokens).

   TTL:
   - Popular products: 24 hours.
   - Daily offers: 24 hours.
   - User data: 30 minutes.

8 Data System Architecture

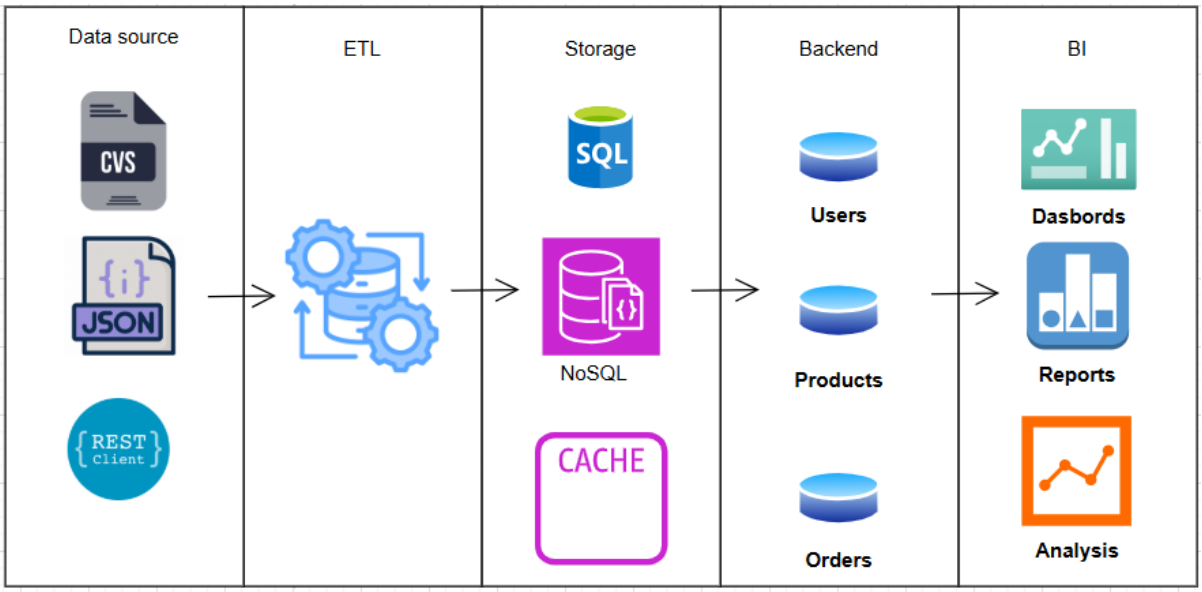

Figure 6: Data system architecture

The proposed architecture for the NexuxBuy e-commerce platform's database management system directly addresses the project's functional and non-functional needs and has been designed based on a set of realistic assumptions that allow for simulating operating conditions of a large-scale platform. First, a data acquisition phase is contemplated from multiple sources, such as CSV files, JSON structures, and REST services. Although these formats reflect typical information input channels in a real e-commerce environment, the data used in this project will be synthetic, that is, artificially generated to simulate real-life behaviors and structures of users, sellers, products, and orders. This decision allows for the validation of the architectural design without the need to access confidential or production data, facilitating volume, variety, and quality control during development.

CSV files will be used, for example, to represent bulk product uploads by sellers; JSON files will be used to represent configurations or other semi-structured data; and REST clients will simulate connections to external APIs, such as identity validators or payment gateways. This diversity allows for modeling a flexible and realistic e-commerce ecosystem, where data comes from multiple systems and actors.

The next layer is the ETL (Extract, Transform, Load) module, which plays a fundamental role in data integration and standardization. Its function is to extract information from different sources, transform it to ensure its quality and integrity, and load it into storage systems. This is especially important given the simulated volume: 600,000 users, 48,000 sellers, 288,530 products, and 3 million orders, creating an environment in which data cleansing, validation, and transformation are essential to ensure system reliability.

Regarding storage, the architecture uses a hybrid strategy that combines SQL, NoSQL, and caching databases. The relational database (SQL) is responsible for managing critical structured information such as users, orders, and entity relationships. This is key to ensuring consistency and secure transactions in operations such as registrations, purchases, and payments. On the other hand, the NoSQL database is used to store semi-structured or unstructured information, such as product images, questions and answers, recommendations, and reviews, whose structure can vary and whose access must be fast and scalable. Finally, the use of caching systems accelerates the most frequently read queries, such as product searches, featured lists, or recently viewed information, optimizing the response in high-load scenarios, such as those generated by 320,000 daily interactions.

The backend is organized into three main services: Users, Products, and Orders, allowing for a clear separation of responsibilities. This facilitates system scalability and maintenance, in addition to reflect-

ing the most important functions within the platform. This modularity is essential to properly manage the interaction of hundreds of thousands of users and sellers, and to ensure the correct processing of millions of records without bottlenecks.

Finally, the architecture includes a Business Intelligence (BI) layer composed of dashboards, reports, and analysis modules. This layer is designed to generate strategic value from the collected data. For example, a salesperson will be able to view real-time product performance, sales volume, and customer behavior in each country (Colombia, Mexico, and Argentina), while an administrator will be able to identify global patterns and make decisions based on historical data from the 8.9 million order lines. This not only improves decision-making but also enriches the user experience by allowing them to receive personalized recommendations and intelligent support.

8.1 Components and tools

The main components that make up the data management system for the e-commerce platform are detailed below, structured into five main modules: Data Source, ETL, Storage, Backend, and Business Intelligence (BI).

- Data Source:
  This module represents the different formats and channels from which the system obtains data. In the context of the project, all data will be synthetic, but real formats will be used to simulate information input.
  - CSV Files: Flat files containing tabular data. Used to simulate bulk uploads of products, users, or inventory, partially imported by sellers.
  - JSON Files: Structured files in a semi-structured format, used to represent system configurations, API responses, or complex data structures such as catalogs or promotions.
  - REST Client: Simulates the connection to external APIs, such as payment gateways, geolocation services, or identity validators. Allows the system to be prepared for real integrations in future stages.
- ETL:
  Since the project relies on realistic but simulated datasets, the ETL (Extract, Transform, Load) process is specifically designed to ingest structured files (e.g., CSV, JSON) representing key business entities such as orders, users, products, reviews, and marketing campaigns. These datasets are either manually curated or synthetically generated to reflect the behavior and structure of a real e-commerce platform. This setup allows for controlled testing of the data system architecture without relying on external, real-time sources. In the extraction phase, the system periodically ingests data from pre-built local repositories. Each dataset is categorized (e.g., orders.csv, products.json, reviews.csv) and versioned to simulate data updates over time.
  During the transformation phase, raw datasets undergo cleansing, type normalization, enrichment (e.g., calculating total order value, classifying review sentiment), and structural adaptation to fit relational (PostgreSQL) and NoSQL (MongoDB) models. For example, denormalized product data is split into atomic tables (products, categories, stock) for PostgreSQL, while user behavior records are reformatted into JSON documents for MongoDB. In this case, Python (with Pandas) or SQL scripts are used for the transformation logic due to their flexibility and simplicity for small and medium data volumes.
  Finally, in the loading phase, the transformed data is inserted into the appropriate systems: core business data (orders, users, payments) in PostgreSQL, and analytical or behavioral data (search history, reviews, Q&A threads) in MongoDB. This hybrid load ensures transactional consistency and analytical readiness. The custom ETL pipeline is essential for simulating realistic data flows and testing the architecture's ability to handle diverse data types. It supports incremental loads and repeatable testing, crucial in academic or prototyping environments. Furthermore, this simulated setup maintains the architectural integrity of a real production e-commerce system.
- Storage:
  This layer contains data persistence solutions, segmented by type and purpose.
  - SQL Database:
    For each transactional operation, such as registering users, processing orders, or updating

inventory, the system uses PostgreSQL as the primary relational database engine. This technology was selected for its strong adherence to ACID (Atomicity, Consistency, Isolation, and Durability) properties, ensuring integrity and reliability in high-volume and high concurrency scenarios. Additionally, PostgreSQL offers advanced capabilities such as native support for semi-structured structures (e.g., JSON), extensibility through user-defined functions, and horizontal read scaling options using replicas

  - NoSQL Database:
    To efficiently manage semi-structured and constantly evolving data, the system incorporates MongoDB, a document-oriented NoSQL database designed for flexibility and scalability. In the context of NexusBuy, an e-commerce platform, MongoDB is particularly well-suited for storing user-generated content such as product reviews, search histories, Q&A interactions, and marketing campaign metadata. Its schemaless model allows the database to dynamically adapt to variations in data structure without requiring costly migrations or rigid constraints, which is essential for capturing real-time behavioral data in a variety of formats. Additionally, MongoDB supports horizontal scaling through sharding, enabling it to handle large volumes of non-uniform and write-intensive data, typical of user interaction logs.
  - Cache:
    To optimize the performance of read-intensive operations, the architecture incorporates Redis as an in-memory cache layer. Redis is a high-performance key-value store that operates entirely in RAM, enabling sub-millisecond response times for frequently accessed data. This makes it particularly well-suited for scenarios where low latency and high throughput are crucial, such as generating dynamic product listings, managing active user sessions, or delivering real-time promotional content. By offloading repeated read operations from the main database, Redis helps improve scalability and reduce the transactional system load. Its support for configurable expiration policies (TTL, time-to-live, to be defined later), persistence options, and atomic operations aligns with modern distributed system design practices.

- Backend:
  The Backend component, within the database management system, acts as an intelligent intermediary between users and storage layers, executing the necessary operations to ensure integrity, consistency, and efficiency in data access. It is comprised of logically decoupled services that encapsulate the main operations on the data model entities. The User service manages the writing and reading of records associated with authentication, profiles, and roles, ensuring that the data complies with uniqueness constraints and relationships with other entities. The Product service interacts directly with relational and non-relational databases to store and retrieve structured information (such as price, stock, and category) and semi-structured information (such as images, reviews, or FAQs), applying filters and efficient pagination. The service orchestrates complex transactions involving multiple related tables, ensuring atomicity through mechanisms such as commit/rollback and referential integrity validation. These services use optimized SQL queries, indexed access, and NoSQL-specific drivers, allowing business logic to execute consistently and in alignment with the physical and logical design decisions of the database.

- Business Intelligence:
  The business intelligence layer is responsible for converting data into useful information for decision-making.

  - Dashboards: Real-time visual dashboards for buyers, sellers, and administrators that display key indicators such as sales, inventory, outstanding questions, and more.
  - Reports: Periodic reports (daily, weekly, monthly) generated from stored data, useful for deeper analysis or audits.

These components are interconnected to enable a continuous flow of operational and analytical data. Data from user actions such as browsing and purchases is processed by backend services, stored in the corresponding databases, and periodically transformed and analyzed using ETL and BI tools.

8.2 Data flow

- ETL Process Flow
  The data flow in the ETL process begins with the extraction of structured files (CSV or JSON)

containing synthetic information, such as products, users, orders, and campaigns. These files are extracted from local repositories. In the transformation phase, the data is cleansed, normalized, and adapted: entities are separated, formats are converted, and derived fields are generated (e.g., the total value of a purchase or the rating of a review by category). Subsequently, in the loading phase, the data is inserted into its respective destinations: PostgreSQL for transactional information and MongoDB for behavioral data or user-generated content. This flow allows the architecture to remain functional and representative of the real world without relying on real-time data sources.

- Flow between Backend, PostgreSQL, and Redis
  When the backend needs to perform critical operations such as entering an order or updating inventory, it writes directly to PostgreSQL, ensuring consistency through its ACID properties. At the same time, to improve performance in subsequent queries, the Redis layer can be updated with newly processed data (for example, new product stock). In scenarios where rapid access to frequently queried data is required, such as lists of best-selling products, the backend first checks Redis. If the information is not found there or is outdated, it queries PostgreSQL, retrieves the data, and stores it back in Redis with a configured expiration policy. This flow optimizes response times without compromising the integrity of the underlying data.

- Flow between Backend and MongoDB
  The backend also interacts with MongoDB to store and query user-generated data that doesn't require a rigid structure. For example, when a new product review or user inquiry (Q&A) is received, the backend structures the content as a JSON document and saves it to MongoDB. This data can also be queried by analysts or other services that need to study customer behavior, search patterns, or campaign interactions. This flow leverages MongoDB's flexibility to store heterogeneous and constantly evolving information without affecting the core relational model.

- Flow to the BI Module
  Once the data has been stored in PostgreSQL and MongoDB, the Business Intelligence (BI) system accesses these sources to generate reports, dashboards, and interactive visualizations. Metabase connects directly to both databases to view metrics such as sales by category, review trends, most visited products, or search trends. This data is presented to users such as administrators or analysts, who can explore it without advanced technical knowledge. The flow of data from the databases to Metabase can be scheduled periodically or triggered on demand, allowing for both real-time visualizations and historical reports.

9 Information Requirements

An NexusBuy system built with FastAPI must manage and integrate information from both relational and non-relational sources to ensure operational continuity, improve customer experience, and support strategic decisions. Below are the key types of information handled by the system and their technical origins.

9.1 Sales & Financial Reporting

- **Gross Merchandise Volume (GMV) by Seller**
  - *Data Source:* PostgreSQL (Transactional)
  - *Schema Source:* `sellermanagement.stores`, `orderprocessing.orderdetails`
  - *Orchestrated by:* FastAPI backend
  - *Desired Output:*

Seller	Total Sales
`Store A`	$25,400
⋮	⋮

- **Coupon Effectiveness**
  - *Data Source:* PostgreSQL (Transactional)
  - *Schema Source:* `orderprocessing.coupons`, `orderprocessing.orders`
  - *Orchestrated by:* FastAPI backend

- *Desired Output:*
  - Redemption rate: 68%
  - Revenue lift: +$12,400

9.2 User Behavior Analysis

- **Search-to-Purchase Conversion**
  - *Data Source:* MongoDB (Analytical)
  - *Schema Source:* `recommendationmanagement.usersearchhistory`, `orderprocessing.orderdetails`
  - *Orchestrated by:* FastAPI analytics service
  - *Desired Output:*
    1. Searches: 10,240
    2. Purchases: 310 (3.0%)
- **Review Sentiment Analysis**
  - *Data Source:* PostgreSQL (Structured)
  - *Schema Source:* `reviewmanagement.reviews.comment`
  - *Processed by:* External NLP service integrated via FastAPI
  - *Desired Output:*
    - Positive: 68%
    - Negative: 12%
    - Neutral: 20%

9.3 Inventory & Catalog Management

- **Stock-Out Risk**
  - *Data Source:* PostgreSQL (Transactional)
  - *Schema Source:* `productcatalog.products.stock`, `orderprocessing.orderdetails`
  - *Monitored by:* FastAPI background task
  - *Desired Output:*
    - `Product ID 7821`: 12 units left
    - `Product ID 4015`: 8 units left
- **Category Trends**
  - *Data Source:* PostgreSQL (Aggregated)
  - *Schema Source:* `productcatalog.categories`, `orderprocessing.orderdetails`
  - *Processed by:* BI API in FastAPI
  - *Desired Output:*
    - Electronics: +15% MoM
    - Fashion: -3% MoM

9.4 Marketing & Campaign Analytics

- **Campaign ROI**
  - *Data Source:* MongoDB (Documental)
  - *Schema Source:* `publicitymanagement.ads`, joined with `orderprocessing.orders`
  - *Processed by:* FastAPI marketing endpoints
  - *Desired Output:*
    - Campaign A: $4.20 return per $1

- **Keyword Accuracy**
  - *Data Source:* MongoDB (Documental)
  - *Schema Source:* `publicitymanagement.campaign.targetKeywords`, matched with `recommendationmanagement.`
  - *Evaluated by:* FastAPI keyword service
  - *Desired Output:*
    - "Gaming chair": 91% match rate

9.5 Order Fulfillment & Logistics

- **Delivery Timeline**
  - *Data Source:* PostgreSQL (Transactional)
  - *Schema Source:* `shippinglogistic.shipments.shippedAt`, `deliveredAt`
  - *Tracked by:* FastAPI tracking endpoint
  - *Desired Output:*
    - Avg. delivery time: 38.2 hours
- **Carrier Performance**
  - *Data Source:* PostgreSQL (Transactional)
  - *Schema Source:* `shippinglogistic.shipments`
  - *Aggregated by:* FastAPI logistics module
  - *Desired Output:*
    - Carrier X: 98% on-time delivery

10 Query Proposal

At the current stage of the project, we are focusing on the evaluation and validation of technological alternatives, including PostgreSQL for structured data, MongoDB for analytical and semi-structured data, Redis for caching, and FastAPI as the backend orchestration layer. No production implementation has been carried out yet; however, a set of representative queries has been designed to verify the alignment between the proposed data model and expected business logic.

These queries serve as a foundation for validating data access layers, verifying schema integrity, and projecting future reporting and performance analysis strategies. Where applicable, synthetic data is assumed to simulate behavior.

10.1 Sales & Financial Reporting

10.1.1 Gross Merchandise Volume (GMV) by Seller

```
WITH product_sales AS (
  SELECT
    od.product_id,
    SUM(od.unit_price * od.quantity) AS total_sales
  FROM orderprocessing.orderdetails od
  GROUP BY od.product_id
)
SELECT
  s.id,
  s.name AS store,
  SUM(ps.total_sales) AS store_sales
FROM product_sales ps
JOIN productcatalog.products p ON ps.product_id = p.id
JOIN sellermanagement.stores s ON p.store_id = s.id
GROUP BY s.id, s.name;
```

Listing 1: PostgreSQL

Purpose: Calculate total sales per seller to support commission tracking.
Insight: Highlights top-performing stores.

10.1.2 Coupon Effectiveness

```sql
SELECT
  c.code,
  COUNT(o.id) AS redemption_count,
  SUM(o.total_price) AS revenue_impact
FROM orderprocessing.coupons c
JOIN orderprocessing.orders o ON c.id = o.coupon_id
WHERE c.is_active = true
GROUP BY c.id;
```

Listing 2: PostgreSQL

Purpose: Evaluate the financial impact of coupon usage.
Insight: Assists in promotional strategy decisions.

10.2 User Behavior Analysis

10.2.1 Search-to-Purchase Conversion

```python
pipeline = [
  {"$group": {
    "_id": "$search_query",
    "searches": {"$sum": 1},
    "purchases": {"$sum": {
      "$cond": [{"$gt": ["$order_id", None]}, 1, 0]
    }}
  }},
  {"$project": {
    "conversion_rate": {
      "$round": [{"$multiply": [
        {"$divide": ["$purchases", "$searches"]}, 100]}, 2]
    }
  }}
]
result = mongo_db.usersearchhistory.aggregate(pipeline)
```

Listing 3: MongoDB Aggregation in FastAPI

Purpose: Measure the percentage of user searches that lead to purchases.
Insight: Identifies high-intent keywords and search behaviors.

10.2.2 Review Sentiment Analysis

```python
pipeline = [
  {"$project": {
    "positive": {"$cond": [{"$gte": ["$rating", 4]}, 1, 0]},
    "negative": {"$cond": [{"$lte": ["$rating", 2]}, 1, 0]}
  }},
  {"$group": {
    "_id": "$product_id",
    "positive_pct": {"$avg": "$positive"},
    "negative_pct": {"$avg": "$negative"}
  }}
]
result = mongo_db.reviews.aggregate(pipeline)
```

Listing 4: MongoDB Aggregation in FastAPI

Purpose: Estimate customer sentiment toward each product.
Insight: Detects potential reputation issues.

10.3 Inventory & Catalog Management

10.3.1 Stock-Out Risk

```sql
WITH product_demand AS (
  SELECT
    od.product_id,
    AVG(od.quantity) AS avg_daily_demand
```

```
  FROM orderprocessing.orderdetails od
  JOIN orderprocessing.orders o ON od.order_id = o.id
  WHERE o.created_at > CURRENT_DATE - INTERVAL '30 days'
  GROUP BY od.product_id
)
SELECT
  p.id,
  p.title,
  p.stock,
  pd.avg_daily_demand
FROM productcatalog.products p
JOIN product_demand pd ON p.id = pd.product_id
WHERE p.stock < pd.avg_daily_demand * 2;
```

Listing 5: PostgreSQL

Purpose: Identify products likely to run out of stock.
Insight: Enables preventive inventory restocking.

10.3.2 Category Trends

```
pipeline = [
  {"$group": {
    "_id": "$category_id",
    "current_month": {"$sum": "$sales"},
    "previous_month": {"$sum": "$previous_sales"}
  }},
  {"$project": {
    "growth_pct": {
      "$multiply": [
        {"$divide": [
          {"$subtract": ["$current_month", "$previous_month"]},
          "$previous_month"
        ]}, 100
      ]
    }
  }}
]
result = mongo_db.category_performance.aggregate(pipeline)
```

Listing 6: MongoDB Aggregation in FastAPI

Purpose: Analyze sales evolution by category.
Insight: Supports catalog optimization.

10.4 Marketing & Campaign Analytics

10.4.1 Campaign ROI

```
pipeline = [
  {"$lookup": {
    "from": "ads",
    "localField": "_id",
    "foreignField": "campaign_id",
    "as": "ads"
  }},
  {"$unwind": "$ads"},
  {"$project": {
    "cost": "$ads.bid_amount",
    "revenue": {
      "$multiply": ["$ads.click_amount", {
        "$divide": ["$ads.conversion_revenue", 1000]
      }]
    }
  }},
  {"$group": {
    "_id": "$_id",
    "roi": {"$sum": {"$divide": ["$revenue", "$cost"]}}
  }}
]
```

```
result = mongo_db.campaigns.aggregate(pipeline)
```

Listing 7: MongoDB Aggregation in FastAPI

Purpose: Calculate return on ad investment.
Insight: Assists in budget reallocation.

10.4.2 Keyword Accuracy

```
pipeline = [
  {"$project": {
    "keywords": {"$split": ["$target_keywords", ","]},
    "campaign_id": 1
  }},
  {"$unwind": "$keywords"},
  {"$lookup": {
    "from": "usersearchhistory",
    "localField": "keywords",
    "foreignField": "search_query",
    "as": "matches"
  }},
  {"$project": {
    "match_count": {"$size": "$matches"}
  }},
  {"$group": {
    "_id": "$campaign_id",
    "match_rate": {"$avg": "$match_count"}
  }}
]
result = mongo_db.campaigns.aggregate(pipeline)
```

Listing 8: MongoDB Aggregation in FastAPI

Purpose: Evaluate the precision of campaign targeting.
Insight: Improves future keyword planning.

10.5 Order Fulfillment & Logistics

10.5.1 Delivery Timeline

```
SELECT
  EXTRACT(EPOCH FROM (delivered_at - shipped_at))/3600 AS delivery_hours,
  COUNT(*) AS shipments
FROM shippinglogistic.shipments
WHERE delivered_at IS NOT NULL
GROUP BY 1;
```

Listing 9: PostgreSQL

Purpose: Measure average delivery times.
Insight: Identify SLA deviations.

10.5.2 Carrier Performance

```
pipeline = [
  {"$match": {"status": "Delivered"}},
  {"$group": {
    "_id": "$carrier",
    "avg_delivery_hrs": {"$avg": "$delivery_hours"},
    "on_time_rate": {
      "$avg": {
        "$cond": [{"$lte": ["$delivery_hours", 48]}, 1, 0]
      }
    }
  }}
]
result = mongo_db.shipments.aggregate(pipeline)
```

Listing 10: MongoDB Aggregation in FastAPI

Purpose: Benchmark delivery service providers.
Insight: Supports carrier selection decisions.

10.6 Caching Strategy with Redis in FastAPI

10.6.1 Product Recommendations Cache

```
@redis_cache(ttl=3600, key="user_recs:{user_id}")
def get_user_recommendations(user_id: str):
    return generate_recommendations(user_id)
```

Listing 11: Redis-Backed Cache Example

Purpose: Reduce response time for recommendation endpoints.
Benefit: Improves perceived latency for end-users.

10.6.2 Product Detail Caching

```
@redis_cache(ttl=600, key="product:{product_id}", skip_if="stock < 10")
def get_product_detail(product_id: int):
    return fetch_product_with_discount(product_id)
```

Listing 12: FastAPI with Conditional Cache

Purpose: Cache popular product views selectively.
Benefit: Stabilizes load during traffic spikes.

10.7 Technology Integration Strategy

The current phase of the project is focused on evaluating the most suitable technologies for the e-commerce platform architecture. No concrete implementation has been carried out yet. The following components and tools are under active analysis:

- **PostgreSQL:** Proposed as the core transactional database to manage structured entities such as users, roles, addresses, orders, order details, products, categories, coupons, payments, shipments, and their statuses.
- **MongoDB:** Considered for storing non-relational and analytical entities. These include both:
  - Entities migrated from the relational model:
    - `ProductQuestions`, `ProductAnswers` (Q&A)
    - `Conversations`, `Messages` (Chat)
    - `Campaign`, `Ad` (PublicityManagement)
    - `UserSearchHistory` (RecommendationManagement)
    - `ProductImages`, `ProductDiscounts` (ProductCatalog)
  - Aggregated behavioral or analytical logs for categories, search terms, campaign performance, and carrier metrics.
- **Redis:** Evaluated for use as a caching layer to accelerate access to frequently queried endpoints such as product details and recommendation services.
- **FastAPI:** Selected as the candidate web framework for orchestrating API endpoints, backend services, and integration with databases and cache layers.
- **ETL Pipeline:** an ETL pipeline is one option to periodically extract aggregates from PostgreSQL and store them in MongoDB for historical analysis.

11 Concurrency Analysis

Concurrency management is a fundamental aspect in the design of database systems, especially in high-traffic platforms like NexusBuy. Concurrency refers to a system's ability to process multiple operations or transactions simultaneously, or seemingly simultaneously, ensuring data consistency and integrity. In an environment where multiple users access and modify the same data at the same time, it is crucial to implement robust mechanisms to avoid conflicts and ensure that operations are completed correctly and

atomicly.

In NexusBuy, concurrency arises in various scenarios, such as simultaneous purchases of products with limited stock, inventory updates by sellers while buyers interact, massive uploads of product reviews by users, the application of promotions by multiple sellers, and the generation of personalized recommendations in real time for numerous users. Furthermore, highly concurrent read queries, especially for popular products, are critical points for the system. It's important to note that these situations are particularly acute during peak demand dates such as Black Friday, Mother's Day, Christmas, and other specific times, when transaction volume and the number of concurrent users can increase exponentially, requiring an extremely resilient and optimized system.

11.1 Concurrency problems

- **Race Conditions:**
  Problem: A key example is overselling of products when multiple buyers simultaneously attempt to purchase the last item in stock. If transactions are not managed properly, both could see the available stock before the quantity is updated, leading to negative inventory.

  Solution and Technical Justification: Pessimistic locking protocols, specifically row- or table-level locks, will be implemented for critical sections of transactional operations, such as inventory updates. This ensures that only one transaction can modify a product's stock at a time, blocking others until the operation completes and the resource is released. This strategy ensures data integrity, which is paramount to avoiding overselling in an e-commerce environment, especially under the pressure of events like Black Friday.

- **Deadlocks:**
  Problem: These occur when two or more transactions block each other, waiting for resources that the other has locked. For example, Transaction A locks Product X and requests Product Y, while Transaction B locks Product Y and requests Product X.

  Solution and Technical Justification: Consistent locking orders will be established for transactions involving multiple resources. This means that all transactions must acquire locks in a predefined sequence, eliminating the conditions for a wait cycle. Additionally, the system will have a deadlock detection and resolution mechanism that, upon identifying a deadlock, will abort one of the ("victim") transactions to allow the other to proceed, and the aborted transaction will be retried. This is critical for the system's responsiveness, preventing operations from being paralyzed indefinitely, which is critical during peak concurrency.

- **Inconsistent Reads (Dirty Reads, Non-Repeatable Reads, Phantom Reads):**
  Problem: Analytical operations or recommendation modules could read data that has not yet been committed by a transaction, or that has changed between reads, leading to inaccurate results (e.g., recommending a recently out-of-stock product). This is particularly relevant in high-demand periods, where information must be as accurate as possible.

  Solution and Technical Justification: Appropriate SQL isolation levels will be used. For critical transactional operations, such as order processing, an isolation level such as "Read Committed" will be applied to prevent dirty reads, or "Repeatable Read" to ensure that repeated reads within a transaction produce the same data set. For analytical data in a NoSQL database, where eventual consistency is often acceptable in exchange for performance, a more relaxed approach, or optimistic concurrency control (OCC), can be employed. OCC allows transactions to proceed assuming there will be no conflicts, verifying only at commit time. If a conflict is detected, the transaction is rolled back and retried. This strategy is ideal for scenarios with many reads and few writes, such as user interactions and reviews, improving performance by avoiding lock overhead, which is beneficial for the volume of data generated during busy periods.

12 Parallel and Distributed Database Design

The design will adopt a distributed hybrid architecture, combining relational and NoSQL databases, each optimized for its specific workloads and distributed geographically.

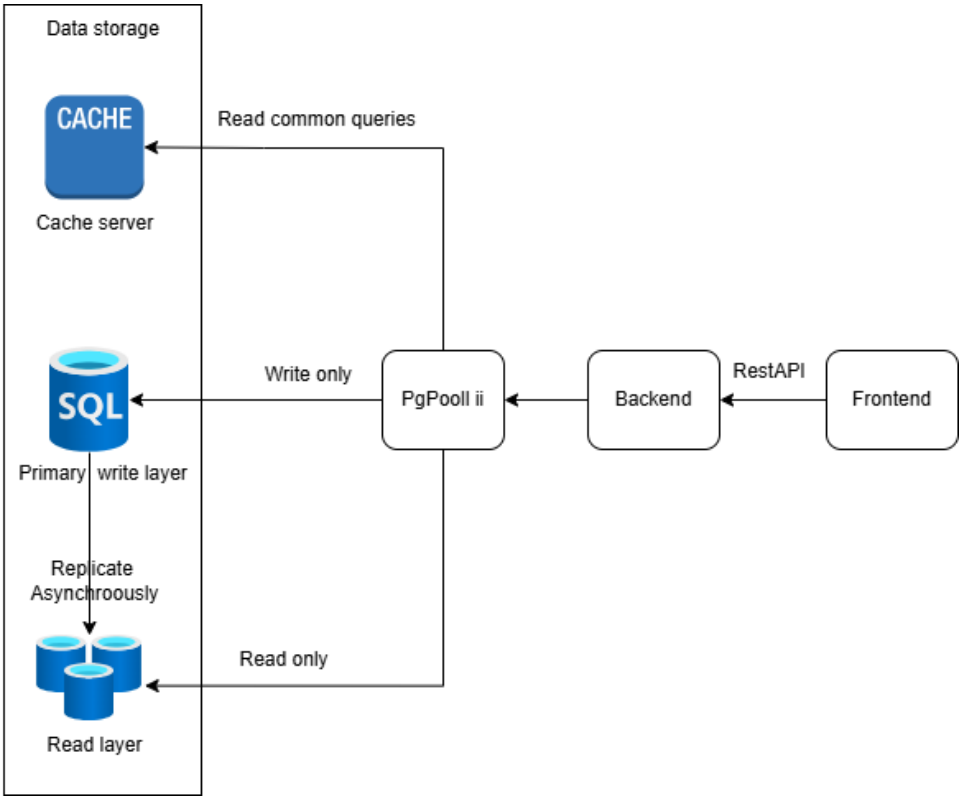

Figure 7: Data system architecture

Transactional Relational Database (SQL) This will be the foundation for core e-commerce operations (user registration, product listing, order processing, payments).

- Distribution Strategy: Regional sharding will be implemented. Data (users, orders, products) will be divided and stored in separate primary database instances for each region (Colombia, Mexico, Argentina). This optimizes the performance of localized queries by reducing the amount of data scanned and improves horizontal scalability to handle projected data growth.

- Replication: Asynchronous replication within and between regions will be used to ensure high availability and disaster recovery. Secondary replicas will handle the read load, freeing up the primary instance for writes, which is crucial for fault tolerance in a multi-location environment.

Analytical Document Database (NoSQL) This database will store unstructured or semi-structured data, such as user reviews, search logs, platform interactions, and personalized recommendation data.

- Distribution Strategy: This database will be globally distributed, with data replication across regions. Its flexible schema and horizontal scalability make it ideal for handling rapidly growing behavioral data.

- Justification: NoSQL databases are ideal for big data and real-time analytics scenarios. This workload separation ensures that complex analytical queries do not impact the performance of transactional operations.

Cache System (Redis) A distributed cache layer will be deployed in front of both databases. This will accelerate access to high-demand data, such as popular product details, user sessions, and recommendations, reducing the load on the primary databases.

- Justification: It is a critical component for improving response times, especially in high-concurrency read scenarios.

13 Performance Improvement Strategies

NexusBuy's design seeks to ensure high availability, rapid query responses, seamless data ingestion, and the ability to perform real-time analytics. To achieve these goals in a large-scale e-commerce system, various performance improvement strategies are employed that are essential to NexusBuy's specific context.

Horizontal Scaling (Sharding/Partitioning)

- Explanation: Instead of the capacity of a single server, more servers will be added to increase the workload and data. In relational databases, this is achieved through sharding, or partitioning data into shards based on a key (e.g., region_id), each on a different server. NoSQL databases typically scale horizontally.
- Technical Justification: Given NexusBuy's large amount of data (600,000 users, 3 million orders) and its multi-regional reach, horizontal scaling is essential. Partitioning by region optimizes queries by ensuring data locality, reducing network latency and overall load. This is vital to handle the projected 25% annual growth in transactions and user interactions.
- Tradeoffs and Challenges: Increases architectural and management complexity. Queries requiring joins between multiple shards can be expensive. Data rebalancing if the participation strategy changes is complex. Maintaining strong consistency across geographically distributed shards can introduce latency.

Data Replication (Asynchronous for Reads):

- Explanation: Multiple copies of the data are created on different servers. For NexusBuy, asynchronous replication will be prioritized for read replicas, thus handling the high volume of read queries and improving availability.
- Technical Justification: Replication is critical to NexusBuy's high availability and fault tolerance. By directing read traffic to the replicas, the load on the primary write instance is reduced, improving its performance. This is crucial for e-commerce with high concurrency in popular product reads. Asynchronous replication is suitable for scenarios where slight inconsistency is acceptable in exchange for higher performance, while synchronous replication would be reserved for highly critical financial transactions that will require immediate consistency.
- Tradeoffs and Challenges: Asynchronous replication can result in "eventual consistency," where replicas may be slightly out of date, requiring careful management for critical data such as inventory. Managing multiple copies of data increases storage costs and administrative overhead. Failover mechanisms must be robust to ensure seamless transitions to a replica in the event of a primary failure.

Caching Strategies

- Explanation: Store frequently accessed data in fast memory (such as Redis) close to the application layer. When a data request is made, the cache is first checked; If present, it is served from there, preventing the database query.
- Technical Justification: The NexusBuy project has already identified caching as a key strategy to accelerate access to high-demand data, reduce the load on the main database, and improve response latency, especially for concurrent read queries on popular products. This directly addresses the need for fast responses and improves the user experience by speeding up personalized recommendations and product details.
- Tradeoffs and Challenges: Cache invalidation, i.e., ensuring that cached data is up to date with the main database, is a significant challenge to avoid displaying outdated information (e.g., an out-of-stock product). It requires efficient memory management and appropriate data eviction policies to avoid out-of-memory errors.

References
Appendix A: Adjustments Based on Feedback

Following the review and feedback received, several structural and content-related modifications were applied to enhance the clarity, consistency, and organization of the document. These changes aim to improve the user experience and better reflect the evolving design of the distributed database system. The following updates were made:

- **Introduction Section:** Expanded to include two critical subsections:
  - **Scope** – Clearly defines the boundaries of the project, outlining the functionalities covered, the technologies involved, and the geographic regions considered (Colombia, Mexico, and Argentina).
  - **Assumptions** – Lists the foundational assumptions used during the research and design of the distributed database system, including data volume estimates, growth projections, technology choices, and behavioral patterns.
- **Preliminary Contextual Section:** Before the technical development of Workshop 1, a new section titled **Description of the NexusBuy Database Management System** was added. This section provides an overview of the platform's goals, architecture, and data components, offering the necessary context to understand the implementation decisions.
- **User Stories:** It was reworked with a card-style layout to improve readability and visual structure, making it easier to associate each story with its functionality and stakeholder. It was written from a customer/user perspective, and the acceptance criteria were improved.
- **Database Architecture:** A justification for the technical decisions of the database architecture is included.
- **Entity-Relationship Model Adjustments:** Based on the evaluation of system requirements and data characteristics, several entities and components were migrated to NoSQL technologies. These adjustments affected multiple parts of the design:
  - **Components Diagram:** Now displays only relational components.
  - **Relationships and Entities:** Updated to reflect the exclusion of non-relational components from the ER model.
  - **Relationship Matrix:** Instead of using an external image, the matrix was integrated as a native LaTeX table for improved accessibility and editing.
  - **Table of Relation Types:** Adjusted to match the refined ER model and the selected database technologies.
  - **First and Second ER Models:** Updated to reflect the division between relational and NoSQL components.